

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Комп'ютерна графіка та обробка зображень»

Робота №5

Тема «Камера»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Туленков А.В.

2025

ЗАВДАННЯ

Завдання на роботу зі сторінки 23:

1. Змініть фігуру та колір.
2. Змініть розмір вікна.
3. Наведіть коментарі до коду.
4. Опишіть алгоритм роботи камери.
5. Знайдіть зайве у коді.
6. Внесіть індивідуальні зміни до коду.

РЕЗУЛЬТАТ

Редагований код

```
#include <stdio.h>
#include <string.h>
#include <cmath>
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

// Нові константи розміру вікна
const unsigned int SCR_WIDTH = 1024;
const unsigned int SCR_HEIGHT = 768;

// Клас, що представляє камеру (точку зору) у 3D-сцені
class Camera
{
public:
    Camera();
    // Повертає матрицю виду, яка перетворює світові координати у
    координати виду
    glm::mat4 getViewMatrix();
    // Обробляє натискання клавіш (W, S, A, D) для руху камери
    void processInput(GLFWwindow *window);

private:
    glm::vec3 cameraPos;        // Позиція камери у світі
    glm::vec3 cameraFront;     // Вектор, куди дивиться камера
    glm::vec3 cameraUp;        // Вектор 'верх' камери (зазвичай (0, 1,
0))
```

```

    float deltaTime;          // Час, що минув з останнього кадру (для
плавності руху)
    float lastFrame;          // Час останнього кадру
};

// Конструктор: встановлення початкових значень
Camera::Camera()
    : cameraPos(glm::vec3(0.0f, 0.0f, 5.0f)), // Камера відсунута від
початку координат
    cameraFront(glm::vec3(0.0f, 0.0f, -1.0f)), // Дивиться вздовж
від'ємної осі Z
    cameraUp(glm::vec3(0.0f, 1.0f, 0.0f)), deltaTime(0.0f),
lastFrame(0.0f) {}

// Обчислює матрицю виду за допомогою GLM::lookAt
glm::mat4 Camera::getViewMatrix()
{
    // lookAt(позиція камери, позиція, на яку дивиться камера (позиція
+ напрямок), вектор "верх")
    return glm::lookAt(cameraPos, cameraPos + cameraFront, cameraUp);
}

// Обробка вводу з клавіатури
void Camera::processInput(GLFWwindow *window)
{
    // Обчислення deltaTime для незалежності руху від частоти кадрів
    float currentFrame = glfwGetTime();
    deltaTime = currentFrame - lastFrame;
    lastFrame = currentFrame;

    // Збільшена швидкість камери (індивідуальна зміна)
    const float cameraSpeed = 5.0f * deltaTime;

    // Рух вперед (W) / назад (S)
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
        cameraPos += cameraSpeed * cameraFront;
    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
        cameraPos -= cameraSpeed * cameraFront;

    // Рух праворуч (D) / ліворуч (A) - "стрейф"
    // Спочатку обчислюємо вектор "праворуч" за допомогою векторного
добутку (cross product)
    // Нормалізуємо, щоб отримати вектор одиничної довжини
    if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
        cameraPos -= glm::normalize(glm::cross(cameraFront, cameraUp))
* cameraSpeed;
    if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
        cameraPos += glm::normalize(glm::cross(cameraFront, cameraUp))
* cameraSpeed;
}

int main()
{
    // Ініціалізація GLFW
    if (!glfwInit())
    {
        fprintf(stderr, "GLFW initialization failed\n");
        return -1;
    }

```

```

}

// Налаштування версії OpenGL (Core Profile 3.3)
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);

// Створення вікна з новим розміром (1024x768)
GLFWwindow *window = glfwCreateWindow(SCR_WIDTH, SCR_HEIGHT,
"Camera Example", NULL, NULL);
if (!window)
{
    fprintf(stderr, "GLFW window creation failed\n");
    glfwTerminate();
    return -1;
}

glfwMakeContextCurrent(window);

// Ініціалізація GLEW
if (glewInit() != GLEW_OK)
{
    fprintf(stderr, "GLEW initialization failed\n");
    glfwDestroyWindow(window);
    glfwTerminate();
    return -1;
}

// Увімкнення тесту глибини (для коректного відображення 3D-
об'єктів)
glEnable(GL_DEPTH_TEST);

// Встановлення області відображення
glViewport(0, 0, SCR_WIDTH, SCR_HEIGHT);

// Код вершинного шейдера
const char *vertexShaderSource = R"(
    #version 330 core
    layout (location = 0) in vec3 aPos;
    uniform mat4 model;
    uniform mat4 view;
    uniform mat4 projection;
    void main() {
        // Обчислення фінальної позиції вершини
        gl_Position = projection * view * model * vec4(aPos, 1.0);
    }
)";

// Код фрагментного шейдера (встановлює зелений колір)
const char *fragmentShaderSource = R"(
    #version 330 core
    out vec4 FragColor;
    void main() {
        // Змінено колір на зелений
        FragColor = vec4(0.0, 1.0, 0.0, 1.0);
    }
)";

```

```

// Компіляція та лінкування шейдерів (логіка залишена без змін)
GLuint vertexShader = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
glCompileShader(vertexShader);
GLuint fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);
GLuint shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
glUseProgram(shaderProgram);
glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);

// Дані для куба (8 вершин)
GLfloat vertices[] = {
    -0.5f, -0.5f, -0.5f, // 0
    0.5f, -0.5f, -0.5f, // 1
    0.5f, 0.5f, -0.5f, // 2
    -0.5f, 0.5f, -0.5f, // 3
    -0.5f, -0.5f, 0.5f, // 4
    0.5f, -0.5f, 0.5f, // 5
    0.5f, 0.5f, 0.5f, // 6
    -0.5f, 0.5f, 0.5f, // 7
};

// Індекси для 12 трикутників, що формують куб (36 індексів)
unsigned int indices[] = {
    0, 1, 2, 2, 3, 0, // Передня
    4, 5, 6, 6, 7, 4, // Задня
    0, 4, 7, 7, 3, 0, // Ліва
    1, 5, 6, 6, 2, 1, // Права
    3, 2, 6, 6, 7, 3, // Верхня
    0, 1, 5, 5, 4, 0 // Нижня
};

// Позиції для трьох кубів (індивідуальна зміна)
glm::vec3 cubePositions[] = {
    glm::vec3( 0.0f, 0.0f, 0.0f),
    glm::vec3( 2.0f, 5.0f, -15.0f),
    glm::vec3(-1.5f, -2.2f, -2.5f)
};

// Створення VAO, VBO та EBO (Index Buffer Object)
GLuint VAO, VBO, EBO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glGenBuffers(1, &EBO); // Додано EBO

glBindVertexArray(VAO);

glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

// Прив'язка індексного буфера

```

```

    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
GL_STATIC_DRAW);

    // Налаштування атрибутів вершин
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 *
sizeof(GLfloat), (GLvoid *)0);
    glEnableVertexAttribArray(0);

    // Відв'язка
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);

    // Отримання location uniform-змінних
    GLint modelLoc = glGetUniformLocation(shaderProgram, "model");
    GLint viewLoc = glGetUniformLocation(shaderProgram, "view");
    GLint projLoc = glGetUniformLocation(shaderProgram, "projection");

    Camera camera; // Створення об'єкта камери

    // Головний цикл рендерингу
    while (!glfwWindowShouldClose(window))
    {
        // 1. Обробка вводу (пух камери)
        glfwPollEvents();
        camera.processInput(window);

        // 2. Рендеринг
        // Темно-сірий колір фону
        glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

        glUseProgram(shaderProgram);
        glBindVertexArray(VAO);

        // 3. Обчислення та передача матриць
        glm::mat4 view = camera.getViewMatrix(); // Матриця виду
        // Матриця проєкції (оновлюємо співвідношення сторін для нового
розміру вікна)
        glm::mat4 projection = glm::perspective(glm::radians(45.0f),
(float)SCR_WIDTH / (float)SCR_HEIGHT, 0.1f, 100.0f);

        glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
        glUniformMatrix4fv(projLoc, 1, GL_FALSE,
glm::value_ptr(projection));

        // Рендеринг трьох кубів
        for(unsigned int i = 0; i < 3; i++)
        {
            // Матриця моделі для кожного куба
            glm::mat4 model = glm::mat4(1.0f);
            model = glm::translate(model, cubePositions[i]); //
Переміщення до позиції

            // Легке обертання для візуалізації 3D
            float angle = 20.0f * (i + 1) + (float)glfwGetTime() *
10.0f;

```

```

        model = glm::rotate(model, glm::radians(angle),
glm::vec3(1.0f, 0.3f, 0.5f));

        glUniformMatrix4fv(modelLoc, 1, GL_FALSE,
glm::value_ptr(model));

        // glDrawElements для рендерингу куба за індексами
        glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_INT, 0);
    }

    // 4. Обмін буферів та події
    glBindVertexArray(0);
    glfwSwapBuffers(window);
}

// Очищення ресурсів
glfwDestroyWindow(window);
glfwTerminate();
return 0;
}

```

Опис алгоритму камери

Алгоритм роботи камери від першої особи залежить від «Матриці виду» (View Matrix).

Матриця виду — getViewMatrix()

Камера використовує функцію `glm::lookAt()`, яка створює матрицю, що імітує камеру. Ця функція вимагає такі вектори:

1. Позиція камери — `cameraPos` — вектор (x,y,z) де знаходиться камера;
2. Цільова позиція — місце, на яке дивиться камера. Обчислюється як `cameraPos+cameraFront`. `cameraFront` — це напрямний вектор камери.
3. Вектор «вгору» -- `cameraUp` — вектор, який визначає, який напрямок є «верхом» для камери. Зазичай (0,1,0).

$$ViewMatrix = lookAt(cameraPos, cameraPos + cameraFront, cameraUp)$$

Зайвий код

При додаванні куба зайвим кодом є:

- дані про старий трикутник:

```
GLfloat vertices[] = {  
    -0.5f, -0.5f, 0.0f,  
    0.5f, -0.5f, 0.0f,  
    0.0f, 0.5f, 0.0f};
```

- зайві заголовки: для коду не потрібен заголовок `<string.h>`, бо він не використовується.